

OAuth 2.0 & JWT Vulnerabilities — Interview Prep Notes

1. Logical Description

OAuth 2.0

- Authorization framework (NOT authentication) — delegates access via tokens instead of credentials
- Actors: Resource Owner, Client, Authorization Server (AS), Resource Server (RS)
- Flow: client → `/authorize` → user consent → auth code → `/token` exchange → access token → RS
- OIDC adds identity layer (`id_token` = JWT) on top of OAuth for authentication

JWT

- Self-contained token: `header.payload.signature` (Base64URL, NOT encrypted — just encoded)
 - Header: `alg`, `typ`, optionally `kid` / `jku` / `x5u`
 - Payload: claims (`sub`, `exp`, `nbf`, `iss`, `aud`, custom claims like `role`)
 - Signature: proves integrity — HMAC (shared secret) or RSA/ECDSA (key pair)
 - Vulns arise when server trusts client-controlled parts (alg, kid, jku) instead of enforcing server-side policy
-

2. OWASP Category & CWE

- **OWASP Top 10 2021:** A07 (Identification & Authentication Failures) — primary; A05 (Security Misconfiguration) — OAuth/JWKS misconfig; A01 (Broken Access Control) — claim tampering → privilege escalation
 - **OWASP API Top 10 2023:** API2 (Broken Authentication), API8 (Security Misconfiguration)
 - **CWE:**
 - CWE-287 — Improper Authentication
 - CWE-347 — Improper Verification of Cryptographic Signature
 - CWE-345 — Insufficient Verification of Data Authenticity
 - CWE-346 — Origin Validation Error (`redirect_uri`)
 - CWE-352 — CSRF (missing `state`)
 - CWE-613 — Insufficient Session Expiration
 - CWE-522 — Insufficiently Protected Credentials (token storage)
 - CWE-330 — Use of Insufficiently Random Values (weak `state` /nonce)
-

3. Common Variants

OAuth:

- `redirect_uri` manipulation / bypass (open redirect chaining)
- Missing/weak/predictable `state` → CSRF on OAuth login
- Authorization code interception (no PKCE, public clients)
- Implicit flow token leakage (Referer header, browser history, logs)
- Client secret exposure (mobile/SPA hardcoded)
- Scope over-grant / scope escalation

- OAuth account-linking takeover (unverified email auto-link)
- Refresh token theft/replay (no rotation/reuse detection)

JWT:

- `alg: none` bypass
 - Algorithm confusion (RS256 → HS256, sign with public key as HMAC secret)
 - Weak/hardcoded HMAC secret (brute-forceable)
 - `kid` header injection (path traversal / SQLi to load attacker key)
 - `jku` / `x5u` header injection (SSRF / attacker-hosted JWKS)
 - `jwk` header injection (embed attacker's own public key)
 - Missing signature verification entirely
 - Missing `exp` / `nbf` check → token never expires
 - Missing `aud` / `iss` check → token confusion across services
 - Sensitive data in payload (mistaking encoding for encryption)
-

4. Attack Surface / Entry Points

- `/authorize`, `/token`, `/callback` (`redirect_uri`) endpoints
 - `Authorization: Bearer <JWT>` header
 - Cookies storing JWT/session
 - "Login with X" SSO buttons
 - Mobile deep-link custom URI schemes
 - API gateway / microservices trusting JWT claims blindly
 - JWKS endpoint (`/.well-known/jwks.json`)
 - Refresh token endpoint
 - Password-reset / account-linking flows tied to OAuth identity
-

5. Testing Workflow

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/Test
Map OAuth flow	granttype, responsetype, endpoints	Browser proxy walk-through	Proxy history + Logger++	Spider + manual explore	N/A (recon)	Identify <code>code</code> / <code>token</code> response_type
redirect_uri validation	Exact-match enforced?	Change to attacker domain/subdomain	Repeater — swap redirect_uri	Manual request editor	<code>oauth-redirect-uri</code> template if exists	<code>https://evil.com</code> , <code>https://victim.com.evil.com</code> , <code>https://victim.com@evil1.com</code> , <code>/\evil.com</code> , path traversal <code>/callback/..%2f..%2fevil.com</code>
state param	CSRF-able login?	Remove/reuse/predict state, replay auth link	Repeater — drop/reuse state	Manual replay	N/A	Empty state, static/predictable state, cross-session reuse
PKCE absence	Code interceptable w/o verifier	Intercept code, exchange without code_verifier	Repeater on <code>/token</code>	Manual token request	N/A	Omit <code>code_verifier</code> , downgrade to <code>plain</code> challenge
Implicit token leak	Token in URL fragment/Referer/logs	Check browser history, Referer header on 3rd-party resource load	Proxy history search	HUD/Manual	N/A	Load external resource post-redirect, inspect Referer
JWT structure	alg used, claims present	Decode Base64URL manually	Burp JWT Editor extension	ZAP JWT add-on	<code>jwt</code> Nuclei templates (exposed signing)	Decode header/payload
alg:none	Signature enforcement	Strip signature, set <code>alg:none</code>	JWT Editor → "none" attack	Manual header edit	Custom nuclei template	<code>{"alg":"none","typ":"JWT"}</code> + empty sig
alg confusion RS256→HS256	Key confusion	Sign token using server's public RSA key as HMAC secret	JWT Editor "RS256 to HS256"	Manual + jwt_tool via CLI	jwt_tool <code>-x k</code>	<code>openssl</code> extract pubkey, HMAC-sign with it
Weak secret brute-force	Secret strength	jwt_tool/hashcat/john offline crack	Intruder w/ wordlist against Repeater test	N/A	N/A	hashcat mode 16500, rockyou.txt
kid injection	Path/SQLi via kid	Modify <code>kid</code> to <code>../../dev/null</code> , <code>' OR '1'='1</code>	JWT Editor manual header edit	Manual	N/A	<code>kid: ../../dev/null</code> (null-byte signature bypass)
jku/x5u SSRF	External key fetch trust	Point jku to Collaborator-hosted JWKS with own key	Collaborator client + JWT Editor	OAST add-on	N/A	<code>jku: https://attacker.com/jwks.json</code>

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/Test
Claim tampering	Server enforces claims post-verify?	Modify <code>role / sub</code> , test if backend re-validates	Repeater	Manual	N/A	<code>role: admin, sub: victim_id</code>
Expiry/replay	exp/nbf enforced	Reuse token after expiry/logout	Repeater with old token	Manual	N/A	Reuse token +1hr past exp
Refresh token reuse	Rotation/reuse detection	Reuse old refresh token after rotation	Repeater	Manual	N/A	Replay rotated-out refresh token
Scope escalation	Server enforces requested scope	Request excess scope, call scoped API	Repeater	Manual	N/A	<code>scope=read</code> token used on write endpoint

6. Evidence to Collect

- Decoded JWT header/payload before & after tampering
- Request/response pair showing 200 + valid session for forged token
- Burp Repeater screenshots of alg:none / alg-confusion acceptance
- HTTP log/Collaborator interaction proving jku SSRF callback
- Redirect chain screenshot showing code/token landing on attacker domain
- Server response with elevated privilege after claim tampering
- Timestamps proving expired/revoked token still accepted

7. Good PoC

- **JWT alg:none:** Decode token → set `alg:none` → strip signature → send to protected endpoint → 200 OK with authenticated response (screenshot + raw request/response)
- **Account takeover via redirecturi:** *Craft malicious OAuth link with attacker redirecturi* → victim clicks → auth code sent to attacker → attacker exchanges code at `/token` → logs in as victim (full request chain + resulting session)
- Must **close the loop**: not just "alg none accepted" but demonstrable unauthorized action/data access

8. Attack Chains

- alg:none/confusion → privilege escalation → IDOR/BFLA on admin APIs
- redirect_uri bypass → auth code theft → account takeover → stored XSS via profile update
- jku/x5u SSRF → internal network access → cloud metadata endpoint (IMDS) compromise
- Missing state → OAuth login CSRF → session fixation → account linking attack
- Refresh token theft (XSS/log leak) → persistent silent account takeover
- Weak JWT secret reuse across microservices → lateral movement across services

9. Common False Positives

- Token expired legitimately — looks like validation bug but isn't
- alg:none rejected but generic error differs from other failures (not exploitable)
- redirect_uri appears permissive client-side but server enforces exact match at `/token`
- Over-privileged claims present in token but re-validated server-side per request (claim ≠ enforced access)
- Self-signed JWT "accepted" structurally but rejected later due to `aud` / `iss` mismatch
- Scanner flags missing `exp` in `idtoken when accesstoken` (which is actually used) has it

10. Prerequisites for Exploitability

- Library allows client-controlled `alg` (no server-side allow-list)
- Short/weak/default HMAC secret
- redirect_uri validated via substring/regex, not exact match
- `state` / PKCE absent for public client
- JWKS/jku not restricted to trusted allow-list
- No mandatory `exp` / `aud` / `iss` validation
- Same signing key/secret reused across services (confusion enables cross-service replay)

11. Attacker Achieves

- Authentication bypass / full account takeover
- Vertical privilege escalation (user → admin)
- Horizontal access to other users' data
- Session hijacking, persistent access via stolen refresh tokens
- SSRF/internal pivot via jku/x5u
- Full API impersonation across microservices (shared-secret confusion)

12. CIA Impact (by variant)

Variant	C	I	A	Example CVSS v3.1 Vector
alg:none bypass	H	H	N	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N (9.1)
alg confusion RS256→HS256	H	H	N	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N (9.8)
Weak HMAC secret brute-force	H	H	N	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N (7.7)
jku/x5u SSRF	H	M	L	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:L (9.4)
redirect_uri bypass (ATO)	H	H	N	AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N (9.6)
Missing state (OAuth CSRF)	L	M	N	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:M... ~ (6.5)
Refresh token replay	H	M	N	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:M/A:N (7.7)

13. Typical CVSS Severity

- Full auth bypass (alg confusion, missing sig check): **Critical, 9.1–9.8**
 - `redirect_uri` → account takeover: **Critical/High, 8.1–9.6**
 - `jku/x5u` SSRF: **Critical, ~9.4** (esp. if reaches cloud metadata)
 - Missing `state` CSRF alone (no token theft): **Medium, ~6.5**
 - Weak secret brute-force (requires compute time): **High, ~7.5**
-

14. Business Impact (Financial App)

- Full account takeover → unauthorized fund transfer/payment initiation
 - Regulatory exposure: PCI-DSS Req 6.2/8 (auth controls), RBI cyber security framework, GDPR Art. 32
 - Mass compromise if JWT secret/signing key shared across services (systemic, not isolated)
 - Reputational damage, mandatory breach disclosure, customer compensation liability
 - Fraud team burden — false transaction disputes from impersonated sessions
-

15. Indicators/Signals

- Repeated JWT verification failures then a sudden success (brute-force pattern)
 - 200 responses for tokens with `alg:none`
 - Outbound requests to unexpected `jku` / `x5u` domains (Collaborator/OAST hits)
 - `redirect_uri` params with non-allow-listed domains in access logs
 - Same refresh token used from multiple IPs/devices simultaneously
 - Tokens with abnormally long/no expiry accepted
 - `state` parameter absent or identical across sessions in logs
-

16. Root Cause (Code-Level)

```
# JWT: algorithm not pinned server-side
jwt.decode(token, key, algorithms=None)    # trusts header's alg
# or
jwt.decode(token, verify=False)            # signature skipped entirely

# OAuth: loose redirect_uri check
if request.redirect_uri.startswith(registered_uri):    # substring match
    proceed()

# Missing state binding
# authorize request generates no state, or state not tied to session server-side

# kid used unsafely
key = read_file(f"/keys/{jwt_header['kid']}")    # path traversal
```

- Core issue: **client-controlled input (alg/kid/jku/redirect_uri) dictates trust decision** instead of server-enforced policy
-

17. Remediation

- Pin allowed algorithms server-side explicitly: `jwt.decode(token, key, algorithms=["RS256"])`
 - Never accept `alg: none`; separate verification paths for HMAC vs RSA/EC — never let key type follow header
 - Exact-match `redirect_uri` allow-list (no wildcard/substring)
 - Mandatory PKCE (S256) for all public clients
 - `state` generated server-side, cryptographically random, bound to session, validated on callback
 - Enforce `exp`, `nbf`, `aud`, `iss` validation on every decode
 - Strong secrets (≥ 256 -bit), prefer asymmetric signing (RS256/ES256) over shared HMAC
 - Restrict `jku` / `x5u` to internal allow-list; never fetch external JWKS dynamically
 - Short-lived access tokens + rotating refresh tokens w/ reuse detection
 - Store tokens in `httpOnly`, `Secure`, `SameSite` cookies — not `localStorage`
-

18. Compensating Controls (if root cause can't be fixed immediately)

- WAF rule blocking `alg:none` / malformed JWT headers
 - Rate-limit `/token` and login endpoints
 - Anomaly detection on token issuance/usage patterns
 - mTLS between internal services (reduces blast radius of stolen JWT)
 - Reduce token TTL as stopgap
 - Monitor/alert on JWKS endpoint access patterns
 - IP/device-binding on refresh tokens
-

19. Recurrence Signal (SDLC Posture)

- No centralized/shared JWT validation library — each team implements independently
 - Inconsistent OAuth client implementation across microservices
 - No security review gate for auth/session design changes
 - Absent threat modeling for third-party OAuth integrations
 - No SAST/DAST coverage specific to JWT/OAuth misconfig class
-

20. Upstream Improvements

- Semgrep/CodeQL taint rule: flag `jwt.decode(..., verify=False)` or missing `algorithms=` param
- Shared internal **auth wrapper library**: enforces alg allow-list, claim validation, key rotation
- Framework-level OAuth client library with PKCE-by-default
- Design checklist item (pre-merge for any auth flow): `redirect_uri` exact match + `state` + PKCE + alg allow-list + claim validation
- Lint rule flagging raw `jsonwebtoken.decode()` usage vs `.verify()`

21. Sample Triage Response

```
Thanks for the report. We've reproduced [JWT alg:none bypass / OAuth
  redirect_uri ATO] on [endpoint]. Confirmed impact: [auth bypass /
  account takeover]. Classified as CWE-347 / OWASP A07:2021,
  CVSS 9.1 (Critical). Escalated to eng team for expedited fix.
  Bounty/severity determination pending internal validation of blast radius.
```

22. Sample Developer Remediation Note

```
Issue: JWT verification accepts `alg:none` / library trusts header-supplied algorithm.
Fix: Pin algorithms explicitly - jwt.decode(token, key, algorithms=["RS256"]).
Do not derive verification method from token header.
Also validate exp/aud/iss on every decode call.
Retest: submit alg:none token → expect 401.
Priority: Critical - patch before next release, no exceptions.
```

23. Real-World CVE/Incidents

- **CVE-2015-9235** — node-jsonwebtoken algorithm confusion (RS256→HS256)
- **CVE-2016-5431 / CVE-2016-10555** — various JWT libs accepting `alg:none`
- **CVE-2022-21449** ("Psychic Signatures") — Java ECDSA signature bypass (empty/zero signature accepted)
- Multiple HackerOne reports — OAuth `redirect_uri` open-redirect → account takeover (Facebook, Slack, Uber login-with-OAuth disclosures)
- Slack bug bounty — missing `state` param → OAuth login CSRF